

Resume programación 702

Certamen 1

Tipos de datos

Nombre	Definicion
String	Variable de tipo texto
Int	Variable de tipo numérico entero
Float	Variable decimal de baja precisión
Double	Variable decimal de alta precisión
Bool	Variable de tipo verdad, puede ser True o False

Operadores lógicos

Nombre	Operador	Definición	Ejemplo
operador Y	and	Operador lógico en donde ambas condiciones deben ser verdaderas para que el resultado sea verdadero	True and True = True
operador o	or	Solo una condición debe ser verdadera para que el resultado sea verdadero	True or False = True
operador no	not	Invierte la condición de verdad	not True = False

Operadores comparativos

Nombre	Operador	Definición	Ejemplo
Igual a	==	Operador que nos devolverá verdadero si dos variables son iguales	1 == 1 = True
Distinto a	!=	Devuelve verdadero si dos valores son distintos	2 != 1 = True
Menor	<	Devuelve verdadero si el valor es estrictamente menor al valor con el que se esta comparando	3 < 4 = True
Mayor	>	Devuelve verdadero si el valor es mayor al valor con el que se esta comparando	4 > 3 = True
Menor o igual	<=	Devuelve True si es el valor es menor o igual	4 <= 4 = True
Mayor o igual	>=	Devuelve verdadero si el valor es mayor o igual	5 >= 5 = True

Operadores matemáticos

Nombre	Operador	Definición	Ejemplo
Suma	+	Suma dos valores en caso de que sean numéricos, si son strings se concatenaran, o sea se agregara uno adelante del otro.	1+1=2

Nombre	Operador	Definición	Ejemplo
Resta	-	Resta de dos valores	2-1=1
Multiplicación	*	Multiplica dos valores	3*2=6
Potencia	**	Nos calcula la potencia de numero	3**2=9
División	/	Divide dos numero, puede devolver un numero decimal(Flotante)	7/2=3.5
División entera	//	Devuelve la parte entera de una división	7/2=3
Modulo	%	Devuelve el resto de una división	7%2=1

Sintaxis

Generales

1. `print(variable)`: Imprime la variable en la terminal
 - Cadena formateada o f-string, es un formato que nos permite introducir variables en un string de una manera mas eficiente, de esta manera nos evitamos usar concatenaciones

```
nombre="Javier"
print(f"Hola mi nombre es: {nombre}") #Hola mi nombre es Javier
```

2. `input()`: Permite que el usuario ingrese datos mediante la terminal.
 - Podemos hacer que el valor que se ingrese sea de un tipo en especifico rodeando el input con el tipo de variable que queremos

```
x=int(input("Ingrese su edad: "))
```

Condicionales

if/elif/else

Este condicional es el mas básico y el mas importante, tiene 3 palabras reservada, todo los condicionales se ejecutaran si la sentencia que se esta evaluando es verdadera:

- `if()`: Es el primer condicional y a diferencia de los otros dos este puede existir por si solo(obligatorio).
- `elif()`: Nos permite agregar condiciones extras, podemos agregar cuantos elif queramos vinculados a un if, pero si o si tiene que haber un if inicial(opcional).
- `else`: Se ejecutara cuando ni el if ni el elif sean verdad(opcional).

```
a = False
b = False
c = False
if(a):
    print("primer if")
elif(b):
    print("segundo condicional")
elif(c):
    print("tercer condicional")
```

```
else:
    print("ultimo condicional")
```

Strings (Cadenas de caracteres)

Los strings son secuencias de caracteres, lo que permite tratarlos como colecciones y realizar operaciones de selección.

Nombre	Definición	Ejemplo
Índice	Accede a un carácter específico según su posición (parte desde 0)	"Hola"[1] = "o"
Slicing (Substring)	Extrae una porción del string usando un rango [inicio:fin]	"Python"[0:2] = "Py"
Salto	Permite extraer caracteres saltándose posiciones [inicio:fin:salto]	"Python"[:2] = "Pt"
Índice Negativo	Accede a los caracteres desde el final hacia adelante	"Hola"[-1] = "a"

Métodos de Strings

Método	Definición	Ejemplo
len()	Devuelve el largo total del string	len("Sol") = 3
.upper()	Convierte todo el texto a mayúsculas	"a".upper() = "A"
.lower()	Convierte todo el texto a minúsculas	"A".lower() = "a"
.count()	Cuenta cuántas veces se repite un carácter o sub-cadena	"banana".count("a") = 3

Bucles (Iteraciones)

Permiten repetir un bloque de código múltiples veces según una condición o una secuencia.

while

Ejecuta el código **mientras** la condición sea verdadera. Se usa cuando no sabemos cuántas veces se debe repetir algo.

```
i = 1
while i <= 3:
    print(i)
    i += 1 # Importante para evitar bucles infinitos
```

for

Se utiliza para recorrer elementos en una secuencia (como un string) o un rango de números definido por `range()`.

Función	Definición	Ejemplo
range(fin)	Genera números desde 0 hasta fin - 1	range(3) -> 0, 1, 2
range(ini, fin)	Genera números desde un inicio hasta fin - 1	range(2, 5) -> 2, 3, 4

```
# Recorrer un string
for letra in "Hola":
    print(letra)

# Recorrer un rango
for x in range(1, 4):
    print(f"Número: {x}")
```

Sentencias de control

- **break**: Rompe el bucle inmediatamente y sale de él.
- **continue**: Salta el resto del código en la iteración actual y pasa a la siguiente.